

第2章 关系数据库

复习思考题

(参考答案)



复习思考题 (1)

1. 试述关系模型的三个组成部分。 ([参考答案](#))
2. 在关系数据模型中，定义并理解下述术语，说明它们之间的联系和区别： ([参考答案](#))
 - ① 域，笛卡尔积，关系，属性，元组，分量
 - ② 表/二维表，列，行，表头
 - ③ 码，候选码，全码，主码，外码
 - ④ 主属性，非主属性
 - ⑤ 关系模式，关系，关系数据库
3. 被称为‘关系’的二维表需要满足哪些性质？ ([参考答案](#))
4. 关系数据库管理系统(SQL语言)中的‘表’和关系模型中的‘关系’有什么区别？ ([参考答案](#))
5. 关于‘关系’的定义 ([参考答案](#))
 - ① 为什么要对关系及关系中的属性进行命名？
 - ② 在一个关系中，是否允许存在同名的属性？
 - ③ 在一个关系中，不同的属性是否允许对应同一个域(domain)？
 - ④ 如何理解关系中的两个‘无序性’：元组的无序性 & 属性的无序性？
 - ⑤ 在一个n元关系中，元组为什么也被称为是‘n元有序组’？
 - ⑥ 什么是关系的‘型’？什么是关系的‘值’？

复习思考题 (2)

6. 请回答以下问题：（参考答案）

- ① 在不同的关系之间，是否允许存在同名的属性？
- ② 在不同的关系之间，是否允许存在值域相同的属性？
- ③ 来自两个不同关系中的同名属性，它们的值域是否一定也相同？
- ④ 来自两个不同关系中的不同名属性，它们的值域是否一定也不相同？

7. 为什么要为关系定义码(Key)？请利用关系和码的定义，给出下述定理的描述性证明：每一个关系系统中都有码。（参考答案）

8. 请按照下述要求分别写出一个例子关系，并说明理由（只需要写出关系模式和语义约束，不需要写出元组集合）（参考答案）

- ① 设计一个关系，存在由多个属性组合构成的码；
- ② 设计一个关系，有多个不同的候选码；
- ③ 设计一个关系，由关系中的所有属性构成该关系的码。

复习思考题 (3)

9. 数据库系统向用户提供了哪四种数据操纵方式？在关系数据模型中，这四种操纵方式又可以被分解为哪五种基本数据操纵功能？分别对应着哪五种关系操作？（[参考答案](#)）
10. 关系模型中的完整性约束规则有哪些？请简要叙述每一条完整性约束规则内容。（[参考答案](#)）
11. 请举例说明，在一个关系中，既有不允许取空值的候选码，也有允许取空值的候选码。（[参考答案](#)）
12. 请举例说明，在参照完整性中，什么情况下在外码属性上允许取空值。（[参考答案](#)）

复习思考题 (4)

13. *设有右图所示的关系 T_1 及其元组集合，在仅有这四个元组的情况下，请找出关系 T_1 的所有候选码。 (参考答案)

T_1	A	B	C	D
	a_1	b_1	c_1	d_1
	a_2	b_3	c_1	d_2
	a_3	b_4	c_2	d_2
	a_4	b_2	c_2	d_1

14. *设有右图所示的关系 T_2 及其元组集合，在仅有这四个元组的情况下，请找出关系 T_2 的所有候选码。 (参考答案)

T_2	A	B	C	D	E
	a_1	b_1	c_1	d_1	e_1
	a_2	b_1	c_1	d_1	e_2
	a_3	b_1	c_2	d_1	e_1
	a_4	b_2	c_1	d_1	e_1

15. *请仿照第13题的关系 T_1 ，自己创建一个新的关系 R_1 ，要求：
- (1) 关系 R_1 中含有 A, B, C, D 四个属性和四个元组； (参考答案)
 - (2) 只有一个候选码 $\{A, B, C\}$.
16. *请仿照第14题的关系 T_2 ，自己创建一个新的关系 R_2 ，要求： (参考答案)
- (1) 关系 R_2 中含有 A, B, C, D, E 五个属性和五个元组；
 - (2) 只有一个候选码 $\{A, B, C, D\}$.

复习思考题 (5)

17. 在关系代数中：(参考答案)

- ① 关系是如何表示的？关系上的数据操纵是如何表示的？
- ② 在关系模型上进行数据操纵，其结果是什么？

18. 关于两个关系之间的并、交、差运算 (参考答案)

- ① 什么样的两个关系，可以在它们之间执行并、交或差运算？
- ② 如果在两个关系之间执行并、交或差运算，那么结果关系的关系模式和结果元组集合分别是什么？
- ③ 为什么说‘交’运算不是关系代数中的基本运算？
- ④ 在什么情况下需要使用‘差’运算？在使用‘差’运算时需要注意什么？
- ⑤ 请比较‘差’运算和‘ $\neq$ ’（不等）比较运算之间的区别。

19. 关于两个关系之间的‘笛卡尔积’ (product) 运算 (参考答案)

- ① 什么样的两个关系，可以在它们之间执行笛卡尔积运算？
- ② 在两个关系之间执行笛卡尔积运算，结果关系的关系模式和结果元组集合分别是什么？
- ③ 笛卡尔积运算在关系数据库访问中有什么实际作用？

复习思考题 (6)

20. 设有一个公司产品零售数据库，其关系模式如下(带下划线的属性是码)： (参考答案)

关系名	属性集	关系模式
顾客	<u>顾客编号</u> , 姓名, 居住城市, 折扣	customers (<u>cid</u> , cname, city, discent)
供应商	<u>供应商编号</u> , 名称, 所在城市, 佣金比例	agents (<u>aid</u> , aname, city, percent)
商品	<u>商品编号</u> , 名称, 库存城市, 库存数量, 单价	products (<u>pid</u> , pname, city, quantity, price)
订单	<u>订单编号</u> , 订购日期, 顾客编号, 供应商编号, 商品编号, 订购数量, 销售金额	orders(<u>ordno</u> , orddate, cid, aid, pid, qty, dollars)

在下述六组关系代数查询表达式中，请分析他们各自查询的目标对象、查询语义和结果集的相互关系。

第1组	$Q_1: \sigma_{pid='p01'}(orders)$	$Q_2: \sigma_{pid \neq 'p01'}(orders)$
第2组	$Q_3: \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第3组	$Q_5: \pi_{cid}(orders) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_6: \pi_{cid}(orders) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第4组	$Q_7: \pi_{cid}(customers) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	$Q_8: \pi_{cid}(customers) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
第5组	$Q_9: orders - \sigma_{pid='p01'}(orders)$	$Q_{10}: orders - \sigma_{pid \neq 'p01'}(orders)$
第6组	$Q_{11}: \pi_{cid}(orders - \sigma_{pid='p01'}(orders))$	$Q_{12}: \pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$

复习思考题 (7)

21. 请用五种基本运算分别写出下列两个扩充运算的推导公式： (参考答案)
- ① ‘自然联接’运算
 - ② ‘除’运算
22. 理解笛卡尔积、 θ -联接、自然联接三者之间的联系和区别，并掌握： (参考答案)
- ① 在何种场景下适合使用自然联接？
 - ② 如何正确使用‘自然联接’运算。
23. 如何理解‘除’运算的语义？请解释其推导公式的含义，并掌握如何正确书写‘除’运算表达式。 (参考答案)
24. 理解 笛卡儿积/ θ -联接/自然联接 和 ‘除’运算之间的区别，并回答： (参考答案)
- ① 在什么情况下适合应用‘除’运算而不是联接运算？请举一例并写出使用除法和不使用除法的两种查询表达式。
 - ② 请列举一个与情况①相似但又不能使用除法的例子，并写出其查询表达式。
25. 什么是关系的自联接？请列举一个使用自联接的例子，并写出查询表达式。
26. 什么是外联接？请列举一个使用外联接的例子，并写出查询表达式。 (参考答案)

复习思考题 (8)

27. 设有一个公司产品零售数据库，其关系模式如下(带下划线的属性是关键字；以订单编号ordno的大小来区分订单的先后，ordno小的在前)。请用关系代数分别写出下述查询。

关系名	属性集	关系模式
顾客	<u>顾客编号</u> , 姓名, 居住城市, 折扣	C (<u>cid</u> , cname, city, discnt)
供应商	<u>供应商编号</u> , 名称, 所在城市, 佣金比例	A (<u>aid</u> , aname, city, percent)
商品	<u>商品编号</u> , 名称, 库存城市, 库存数量, 单价	P (<u>pid</u> , pname, city, quantity, price)
订单	<u>订单编号</u> , 订购日期, 顾客编号, 供应商编号, 商品编号, 订购数量, 销售金额	O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

(1) 查询满足条件的顾客的编号cid:

- ① 只购买过 p01 和 p02 两种商品;
- ② “只购买过商品p01” 或者 “只购买过商品p02”;
- ③ 只购买过一种商品。

(参考答案)

(2) 查询满足条件的供应商的名字aname: 没有销售过商品;

(3) 查询满足条件的供应商的编号aid:

- ① 只向供应商自己所在城市中的顾客销售过商品;
- ② 在所有有顾客的城市中都向客户销售过商品;
- ③ 向供应商自己所在城市中的所有顾客都销售过商品。

复习思考题 (9)

关系名	属性集	关系模式
顾客	<u>顾客编号</u> , 姓名, 居住城市, 折扣	C (<u>cid</u> , cname, city, discent)
供应商	<u>供应商编号</u> , 名称, 所在城市, 佣金比例	A (<u>aid</u> , aname, city, percent)
商品	<u>商品编号</u> , 名称, 库存城市, 库存数量, 单价	P (<u>pid</u> , pname, city, quantity, price)
订单	<u>订单编号</u> , 订购日期, 顾客编号, 供应商编号, 商品编号, 订购数量, 销售金额	O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

- (4) 在每一个供应商自己的所有订单中，查询销售金额dols最高的订单，结果返回供应商编号、该供应商的销售金额最高订单的订单编号和销售金额。
- (5) 查询每一个顾客的第一份订单和最后一份订单（在第①和第②小题中，不考虑没有订单和只有一份订单的顾客；在第③小题中，需要返回所有顾客的查询结果）
- ① 结果关系中只有 cid, ordno, orddate 三个属性；
 - ② 结果关系中有顾客编号cid, 第一份订单的ordno和orddate, 最后一份订单的ordno和orddate等五个属性；
 - ③ *结果关系的关系模式同上面的第②小题，每一个顾客返回一条查询结果。如果是没有订单的顾客，结果元组上只有该顾客的编号；如果是只有一份订单的顾客，那么这份订单就作为该顾客的最后一份订单，他的第一份订单信息为空。（提示：使用外联接）

复习思考题 (10)

关系	属性集	关系模式
学生	<u>学号</u> , 学生姓名, 就读院系, 年级	S (<u>sno</u> , sname, dept, grade)
课程	<u>课程号</u> , 课程名, 开课院系, 课程类型	C (<u>cno</u> , cname, dept, opt)
教师	<u>教师工号</u> , 教师姓名, 工作院系	T (<u>tno</u> , tname, dept)
选课	<u>学号</u> , <u>课程号</u> , 授课教师工号, 成绩, 修读年份	L (<u>sno</u> , <u>cno</u> , tno, score, years)

28. 设有一个如上所示的学生选课数据库，其中：带下划线的属性是关键字；课程类型分为‘必修’和‘选修’；同一门课同一个学生只能有一条选课记录；成绩采用百分制。请用关系代数写出下述查询。

查询满足下述条件的学生的学号和姓名：

(参考答案)

- ① 只修读过自己就读院系开设的课程；
- ② 所有的修读课程成绩都合格（成绩 ≥ 60 ）；
- ③ 修读过‘计算机’系开设的所有‘必修’课程；
- ④ 修读了自己就读院系开设的所有‘必修’课程且在这些必修课程上的成绩都合格；
- ⑤ 2019级并且还没有修读通过自己就读院系开设的所有‘必修’课程。（存在自己就读院系开设的‘必修’课程，该同学还没有修读，或者虽然修读但成绩没达到合格。）

参 考 答 案





1. 试述关系模型的三个组成部分。

答：关系模型包括三个组成部分：数据结构、关系操作和关系的完整性。

- 数据结构：采用单一的数据结构‘关系’。关系是给定域的序列 $D_1, D_2, \dots, D_n$ (其中可能存在相同的域)之间的笛卡尔积 $D_1 \times D_2 \times \dots \times D_n$ 的有限子集。现实世界中的实体以及实体间的各种联系均可以用关系来表示。
- 关系操作：关系模型向用户提供查询、插入、删除、修改等两大类、四种数据操纵功能，这些数据操纵功能又可以通过关系上的并、交、差、笛卡尔积、选择、投影、连接、除法等关系操作来表示。
- 关系的完整性：关系模型具有三类关系完整性规则的定义和检查功能，分别是：
 - ① 实体完整性规则：实体完整性即关系中的元组唯一性，在关系数据库中，可以通过在基表中定义主码来确保基表中行（元组）的唯一性。
 - ② 参照完整性规则：参照完整性是指关系之间的引用约束，在关系数据库中，通过外码来定义基表之间的引用关系，参照完整性规则是指：外码属性的取值只能是以下两种情况之一：取空值，或者等于被参照关系S中某个元组的主码值。
 - ③ 用户定义完整性规则：针对某一具体关系数据库的约束条件，反映某一具体应用所涉及的数据必须满足的语义要求。



2. 在关系数据模型中，定义并理解下述术语，说明它们之间的联系和区别：

① 域，笛卡尔积，关系，属性，元组，分量

- ‘域’是一组具有相同数据类型的值的集合。
- 给定一组域 $D_1, D_2, \dots, D_n$ ，它们的笛卡尔积可表示为 $D_1 \times D_2 \times \dots \times D_n$ ，其运算结果是一个由所有符合要求的‘n元组’组成的集合，即： $D_1 \times D_2 \times \dots \times D_n = \{ (d_1, d_2, \dots, d_n) \mid d_i \in D_i, i=1,2,\dots,n \}$
- 给定一个域的序列 $D_1, D_2, \dots, D_n$ (其中可能存在相同的域)，其笛卡尔积 $D_1 \times D_2 \times \dots \times D_n$ 的一个有限子集叫做在域 $D_1, D_2, \dots, D_n$ 上的‘关系’。
- 关系中一个域对应的列，也被称为是关系的一个‘属性’；关系中的元素 $(d_1, d_2, \dots, d_n)$ 被称为是‘元组’；元组中每个元素 $d_i (i=1, 2, \dots, n)$ 被称为是一个‘分量’。

② 表/二维表，列，行，表头

- 表(table)：由行和列组成的二维表格结构，也被称为‘二维表’；
- 表中的每一列，被称为是该表的一‘列’(column)；
- 除了第一行外，表中的每一行被称为是一‘行’(row)。
- 在表的第一行，存放的是每一列的列名。由表中每一列的列名所组成的集合（即：二维表的第一行），被称为是该表的‘表头’(table heading)。



③ 码，候选码，全码，主码，外码

- 若关系中的某一属性组的值能唯一地标识一个元组，而其所有的真子集都不能，则称该属性组为关系的‘候选码’，简称‘码’。
- 如果一个关系的候选码是由该关系中的所有属性组成，这样的候选码又被称为‘全码’（All-key）。
- 在关系数据库管理系统中，创建基本关系对应的基表时，可以选择该关系的一个候选码作为基表的主码来定义，这个被选中的候选码被称为是‘主码’。
- 设 F 是基本关系 R 的一个属性组，但不是关系 R 的码。如果 F 与基本关系 S 的主码 K_s 相对应，则称 F 是关系 R 的外码。

④ 主属性，非主属性

- 候选码中的诸属性称为该关系的‘主属性’（Prime attribute）
- 不包含在任何候选码中的属性称为该关系的‘非主属性’（Non-Prime attribute）或‘非码属性’（Non-key attribute）

⑤ 关系模式，关系，关系数据库

- 关系模式是对关系的描述，是静态、稳定不变的；
- 关系是元组的集合，是关系模式在某一时刻的状态或内容，是动态的、随时间不断变化的；
- 在一个给定的应用领域中，所有关系的集合构成一个关系数据库。



3. 被称为‘关系’的二维表需要满足哪些性质？

答：需要满足以下六条性质的有限关系：

① **列是同质的 (Homogeneous)**

每一列中的分量是同一类型的数据，来自同一个域。

② **不同的列可出自同一个域**

其中的每一列称为一个属性；

不同的属性要给予不同的属性名。（属性名的唯一性）

③ **列的无序性 (属性的无序性)**

列的排列顺序无所谓，列与列之间的次序可以任意交换；
可通过属性名指定需要访问的列。

④ **行的唯一性 (元组的唯一性)**

任意两个元组在‘候选码’属性上的取值不能相同。

⑤ **行的无序性 (元组的无序性)**

行的排列顺序无所谓，行与行之间的次序可以任意交换；
可通过指定行在列上的取值来确定需要访问的行。

⑥ **分量必须取原子值**

每一个分量都必须是不可分的数据项。



4. 关系数据库管理系统(SQL语言)中的‘表’和关系模型中的‘关系’有什么区别？

答：只有完全满足关系上的六条性质的有限二维‘表’，才能被称为是‘关系’。这六条性质分别是：

- ①列的同质性；
- ②列的唯一性（属性名的唯一性）；
- ③列的无序性；
- ④元组的唯一性；
- ⑤元组的无序性；
- ⑥分量（属性值）的原子性。



5. 关于‘关系’的定义

- ① 为什么要对关系及关系中的属性进行命名?
- ② 在一个关系中, 是否允许存在同名的属性?
- ③ 在一个关系中, 不同的属性是否允许对应同一个域(domain)?
- ④ 如何理解关系中的两个‘无序性’: 元组的无序性 & 属性的无序性?
- ⑤ 在一个n元关系中, 元组为什么也被称为是‘n元有序组’?
- ⑥ 什么是关系的‘型’? 什么是关系的‘值’?

答:

- ① 是为了在一个关系数据库中区分不同的关系, 以及在同一个关系中区分不同的属性。
- ② 在同一个关系中, 不允许有同名的属性。
- ③ 在同一个关系中, 不同的属性可以对应同一个域(domain)。
- ④ ‘无序性’主要体现在关系的表示和关系操作上: 关系模式被表示为‘属性的集合’, 关系被表示为‘元组的集合’; 在关系数据模型中, 只能根据关系名和属性名来确定被访问的关系和属性、根据元组的取值使用‘选择’运算来确定被访问的元组, 不能根据属性或元组的排列次序来访问关系。
- ⑤ 为了确定一个元组中的各个属性值分别对应着关系中的哪一个属性, 元组中的属性值需要按照各自对应的属性在关系模式中的定义顺序来安排, 因而也被称为是一个‘n元有序组’。
- ⑥ 关系模式是一个关系的‘型’, 关系中的元组集合是关系的‘值’。



6. 请回答以下问题：

- ① 在不同的关系之间，是否允许存在同名的属性？
- ② 在不同的关系之间，是否允许存在值域相同的属性？
- ③ 来自两个不同关系中的同名属性，它们的值域是否一定也相同？
- ④ 来自两个不同关系中的不同名属性，它们的值域是否一定也不相同？

答：

- ① 在不同的关系之间，允许存在同名的属性。这些分属于不同关系的同名属性可以是‘同名同义’，也可以是‘同名异义’。
- ② 在不同的关系之间，允许存在值域相同的属性。
- ③ 来自两个不同关系中的同名属性，它们的值域domain和语义meaning不一定相同。
- ④ 来自两个不同关系中的不同名属性，它们的值域可能相同也可能不同。



7. 为什么要为关系定义码(Key)? 请利用关系和码的定义, 给出下述定理的描述性证明: 每一个关系中都有码。

答: 若关系中的某一属性组的值能唯一地标识关系中的一个元组, 而其所有的真子集都不能, 则称该属性组为关系的‘码’。

关系是其各个属性域的笛卡尔积的一个有限子集, 关系中的元组满足唯一性。为一个关系定义码, 可以简化‘元组唯一性’约束检查(不需要比较所有属性的取值), 同时也为‘联系’在关系模型中的表示提供方便。

‘码’是关系内在固有的特性, 每一个关系中都有码。其理由如下:

① 设当前关系为 R , 由 R 中的所有属性构成属性组 K , 则属性组 K 的值能唯一标识关系中的一个元组, 即属性组 K 满足下述性质★:

【性质★】对于关系 R 中的任意两个元组 t_1 和 t_2 , 都有 $t_1[K] \neq t_2[K]$

(至少存在一个属性 $A_i \in K$ 且 $t_1[A_i] \neq t_2[A_i]$)

② 如果 K 的所有真子集都不具备上述的性质★, 则 K 是关系 R 的码, 转步骤④; 否则

③ 至少能够找到 K 的一个真子集 S , 且 S 也满足性质★; 令 $K:=S$ 并返回步骤②;

④ 属性组 K 就是关系 R 的一个码。



8. 请按照下述要求分别写出一个例子关系，并说明理由（只需要写出关系模式和语义约束，不需要写出元组集合）

① 设计一个关系，存在由多个属性组合构成的码；

答：

设有一个高铁旅客出行记录关系 R_1 ，用于每一天全国高铁的旅客乘坐出行情况，关系模式是：

$R_1(\text{旅客身份证号}, \text{高铁车次}, \text{乘车日期}, \text{上车车站名}, \text{下车车站名})$

其中的数据约束有：

- 旅客身份证号、车站名分别是旅客、高铁站的标识属性；
- 每个车次每天只开行一班；
- 同一车次的高铁，每名旅客每天最多只能乘坐一次。

关系 R_1 的一个候选码是 $(\text{旅客身份证号}, \text{高铁车次}, \text{乘车日期})$ 。

(该关系也有其他的候选码)



8. 请按照下述要求分别写出一个例子关系，并说明理由（只需要写出关系模式和语义约束，不需要写出元组集合）

② 设计一个关系，有多个不同的候选码；

答：

设有一个车队车辆出行管理关系： R_2 (车牌号，驾驶员编号，出发时间，返回时间)

其中：

- 车牌号、驾驶员编号分别是车辆、驾驶员的标识属性；
- 每辆车每次出行只安排一名驾驶员；
- 车辆的出发时间和返回时间是包含年月日时分秒的timestamp值。

该关系有四个候选码，分别是：

(车牌号, 出发时间)

(车牌号, 返回时间)

(驾驶员编号, 出发时间)

(驾驶员编号, 返回时间)



8. 请按照下述要求分别写出一个例子关系，并说明理由（只需要写出关系模式和语义约束，不需要写出元组集合）

③ 设计一个关系，由关系中的所有属性构成该关系的码。

答：

➤ 设有一个课程管理关系，其关系模式是 R_3 (课程, 教材, 授课教师)

➤ 其中：

- 一门课程可以有多本教材和多位授课教师
- 一本教材可以为多门课程所用
- 一位教师也可以担任多门课程的授课任务。

➤ 该关系的关键字就是由关系中的所有三个属性联合构成的。



9. 数据库系统向用户提供了哪四种数据操纵方式？在关系数据模型中，这四种操纵方式又可以被分解为哪五种基本数据操纵功能？分别对应着哪五种关系操作？

答：

数据库系统向用户提供了数据查询和插入、删除、修改等两大类、四种数据操纵方式。

在关系数据模型中，上述四种数据操纵方式又可以被分解为单个关系上的数据查询（元组选择、属性投影）、两个关系的合并、单个关系上的元组插入和删除等五种基本数据操纵功能。其中：

- 可以通过单个关系上的元组选择和属性投影、两个关系的合并等操作来实现数据查询功能；
- 可以用元组插入、元组删除操作分别实现数据插入、数据删除功能；
- 可以用旧元组的删除+新元组的插入来实现数据修改功能。

关系模型上五种基本数据操纵功能，可以分别对应着关系代数中的五种基本关系操作，分别是：

- 用‘选择’和‘投影’操作实现单个关系上的元组选择和属性投影；
- 用‘笛卡尔积’操作实现两个关系的合并；
- 用‘并’操作实现元组插入；
- 用‘差’操作实现元组删除。



10. 关系模型中的完整性约束规则有哪些？请简要叙述每一条完整性约束规则内容。

答：关系模型中的关系完整性包括实体完整性规则、参照完整性规则和用户定义的完整性规则。

➤ 实体完整性

- 实体完整性（Entity Integrity）是指关系中元组的唯一性。
- 在关系数据库管理系统中，实体完整性规则是指：若属性A是基本关系（基表）的主码中的属性，则属性A不能取‘空值’。

➤ 参照完整性

- 若属性（或属性组） F 是基本关系 R 的外码，它与基本关系 S 的主码 K_s 相对应（基本关系 R 和 S 不一定是不同的关系），则关系 R 中每个元组在 F 上的取值必须是下列两种情况之一：
 - 空值（ F 的每个属性值均为空值）
 - 等于关系 S 中某个元组的主码值

➤ 用户定义的完整性

- 用户定义的完整性是特定应用领域需要遵循的约束条件，体现了具体领域中的语义约束。



11. 请举例说明，在一个关系中，既有不允许取空值的候选码，也有允许取空值的候选码。

答：设有一个图书借阅关系R(图书编号, 学号, 借阅时间, 归还时间)，其中：

- 图书编号是图书的码，学号是学生的码，借阅时间和归还时间是时间戳类型；
- 一个学生可以同时借阅多本书，也可以在不同时间借阅同一本书。

该关系有两个候选码：（图书编号, 借阅时间）和（图书编号, 还书时间），借阅时间不允许取空值，但还书时间允许出现空值（还书之前）。

12. 请举例说明，在参照完整性中，什么情况下在外码属性上允许取空值。

答：在参照关系中，如果允许其中的某些元组没有引用被参照关系中的元组，那么就应该允许参照关系中的这些元组在外码上取空值。例如：在学生关系(学号, 姓名, 性别, 院系, 专业)中，院系与专业是该关系的两个外码，其中：

- ‘院系’是非空属性，每一位学生都有一个就读‘院系’（或新生书院）
- ‘专业’可以取空值，对于按大类录取的新生，只有在大类分流后才有明确的专业



13. 设有右图所示的关系 T_1 及其元组集合, 在仅有这四个元组的情况, 请找出关系 T_1 的所有候选码。

T_1	A	B	C	D
	a_1	b_1	c_1	d_1
	a_2	b_3	c_1	d_2
	a_3	b_4	c_2	d_2
	a_4	b_2	c_2	d_1

答: 该关系有三个候选码, 分别是: **A, B, (C, D)**

□ 分析推理过程如下:

① 首先考虑由单个属性构成的候选码。

- 属性A: 这四个元组在属性A上的取值是互不相同的, 满足候选码的定义。所以, **单个属性A构成这个关系的一个候选码;**
- 属性B: 同理, **单个属性B也满足候选码的定义, 也是该关系的一个候选码;**
- 属性C: 在这四个元组中, 存在属性C的取值相同的情况, 也就是说, 至少能够找到一对元组, 他们在属性C上的取值是相等的, 因而, 单个属性C组成的属性子集不满足候选码的定义;
- 属性D: 同理, 单个属性D组成的属性子集也不满足候选码的定义

② 考虑由两个属性构成的候选码。因为A和B已经是 T_1 的两个候选码, 它们不可能再是其他候选码的组成成分, 因此只需要考虑CD两个属性, 那就只有(C,D)一种组合。

➤ 经检查, 在关系 T_1 中, (C,D)满足候选码的定义, 理由如下: (1)在关系 T_1 中不可能找到两个不同的元组 u 和 v 且满足 $u[C]=v[C]$ and $u[D]=v[D]$; (2)它的所有真子集都不满足候选码的定义。所以, **(C,D)是关系 T_1 的另一个候选码。**

③ 对于由三个或更多属性组成的属性子集K, K不可能是关系 T_1 的候选码。因为: 这样的属性组K中, 至少真包含着我们已经找到的上述三个候选码之一, 这样的K仅仅是关系 T_1 的一个superkey。



14. 设有右图所示的关系 T_2 及其元组集合，在仅有这四个元组的情况下，请找出关系 T_2 的所有候选码。

T_2	A	B	C	D	E
	a_1	b_1	c_1	d_1	e_1
	a_2	b_1	c_1	d_1	e_2
	a_3	b_1	c_2	d_1	e_1
	a_4	b_2	c_1	d_1	e_1

□ 答：有两个候选码，分别是：**A** 和 **(B,C,E)**

□ 分析推理过程如下（思路与第9题基本相同）：

- ① 首先考虑由单个属性构成的候选码。只有属性A的取值满足候选码的定义，因此，由单个属性构成的候选码是 **A**
- ② 考虑由两个属性构成的候选码。因为A已经是 T_2 的候选码，它不可能是其他候选码的组成成分，因此只需要考虑BCDE之间的两两组合：BC，BD，BE，CD，CE 和 DE。经检查，上述任何一种组合，其取值在关系 T_2 中都不具备‘唯一性’，因而它们都不是关系 T_2 的候选码。
- ③ 考虑由三个属性构成的候选码。不考虑属性A（理由同上），BCDE一共有四种不同的组合：BCD，BCE，BDE 和 CDE。经检查，只有BCE组合的取值在关系 T_2 中具有唯一性，另外三种组合都不具有唯一性。因此，由三个属性构成的候选码是 **(B,C,E)**
- ④ 考虑由四个属性构成的候选码，即BCDE组合。由于BCE已经是 T_2 的候选码，所以BCDE组合不可能是 T_2 的候选码！

另外，经观察发现，在关系 T_2 中属性D的取值是固定不变的，也即是说，任意两个元组在属性D上的取值都是相同的，因此，属性D不可能是关系 T_2 的候选码的组成部分！这样，在分析过程中就不需要考虑属性D，可以极大地简化每一步的分析工作量。



15. 请仿照第13题的关系 T_1 ，自己创建一个新的关系 R_1 ，要求：

- (1) 关系 R_1 中含有A,B,C,D四个属性和四个元组；
- (2) 只有一个候选码 {A,B,C}。

R_1

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_1	c_2	d_1
a_2	b_1	c_2	d_1
a_1	b_2	c_2	d_1

答：（参考答案不唯一） 设计的关系 R_1 如右所示：

□ 设计思考过程如下：

- ① 要求：由前面的ABC三个属性构成该关系的唯一一个‘候选码’；
- ② 在第四个属性D上，值域是一个恒定不变的 d_1 ，这样，属性D就不会是任何候选码的组成部分！
- ③ 前三个属性，每个属性的取值都有‘相等’或‘不相等’两种情况，我们用 a_1 和 a_2 表示两个不同的A值，用 b_1 和 b_2 表示两个不同的B值，……，一共有八种不同的组合： (a_1, b_1, c_1) , (a_1, b_1, c_2) , (a_1, b_2, c_1) , (a_1, b_2, c_2) , ……
- ④ 在上述八种组合中，挑出能够“满足下述要求”的四种组合，再加上属性D组成 R_1 ：
 - a) 对于每一个属性，都存在取值相等的情况（确保单个属性不可能是该关系的候选码）
 - b) 对于任意一种两两组合，都存在取值相等的情况（确保AB，AC和BC中的任意一种组合，都不可能是该关系的候选码）
 - c) 任意两者之间，至少有一个属性上的取值不相等（确保ABC组合的取值是互不相同的，从而由ABC三个属性组合起来构成该关系的候选码）



16. 请仿照第14题的关系 T_2 ，自己创建一个新的关系 R_2 ，要求：

- (1) 关系 R_2 中含有 A, B, C, D, E 五个属性和五个元组；
- (2) 只有一个候选码 $\{A, B, C, D\}$ 。

R_2	A	B	C	D	E
	a_1	b_1	c_1	d_1	e_1
	a_1	b_1	c_1	d_2	e_1
	a_1	b_1	c_2	d_1	e_1
	a_1	b_2	c_1	d_1	e_1
	a_2	b_1	c_1	d_1	e_1

答：（参考答案不唯一）设计的关系 R_2 如右所示：

□ 思考过程如下（思路同第10题）

- ① 要求：由前面的 $ABCD$ 四个属性构成该关系的唯一一个‘候选码’；
- ② 在第五个属性 E 上，值域是一个恒定不变的 e_1 ，这样，属性 E 就不会是任何候选码的组成部分！
- ③ 前四个属性，每个属性的取值都有‘相等’或‘不相等’两种情况，我们用 a_1 和 a_2 表示两个不同的 A 值，用 b_1 和 b_2 表示两个不同的 B 值，……，一共有十六种不同的组合： (a_1, b_1, c_1, d_1) , (a_1, b_1, c_1, d_2) , (a_1, b_1, c_2, d_1) , (a_1, b_1, c_2, d_2) , (a_1, b_2, c_1, d_1) , ……
- ④ 在上述十六种组合中，挑出能够“满足下述要求”的五种组合，再加上属性 E 组成 R_2 ：
 - a) A, B, C, D 中的每一个属性，都存在取值相等的情况；
 - b) 对于 A, B, C, D 中的任意一种两两组合，都存在取值相等的情况；
 - c) 对于 A, B, C, D 中的任意一种‘三个属性’的组合，都存在取值相等的情况；
 - d) 任意两个元组之间，在 A, B, C, D 四个属性中，至少有一个属性上的取值是不相等的（确保 $ABCD$ 组合的取值是互不相同的，从而由 $ABCD$ 四个属性组合起来构成该关系的候选码）



17. 在关系代数中：

- ① 关系是如何表示的？关系上的数据操纵是如何表示的？
- ② 在关系模型上进行数据操纵，其结果是什么？

□ **关系的表示**：在关系代数中，关系是元组的集合。

设有一个 n 目关系 R ， $A_1, A_2, \dots, A_n$ 是关系 R 的 n 个属性， $\text{domain}(A_i)$ 是属性 A_i 的域 ($i=1, 2, \dots, n$)

- 用关系名来表示一个关系： R
- 关系模式的表示： $R(A_1, A_2, \dots, A_n)$
- 元组及分量的表示：设 t 是关系 R 中的一个元组，那么 t 就是一个具有如下形式的 n 元组：
 $(a_1, a_2, \dots, a_n)$ 其中： $a_i \in \text{domain}(A_i)$ for $i = 1, 2, \dots, n$
 元组 t 在属性 A_i 上的取值可以被表示为 $t[A_i]$
- 关系 R 就是如上所示的 n 元组的集合，可以形式化表示为：

$$R = \{ (a_1, a_2, \dots, a_n) \mid a_i \in \text{domain}(A_i) \text{ for } i = 1, 2, \dots, n \}$$

或者表示为： $R = \{ t \mid t[A_i] \in \text{domain}(A_i) \text{ for } i = 1, 2, \dots, n \}$

由笛卡尔积的定义可知： $R \subseteq \text{domain}(A_1) \times \text{domain}(A_2) \times \dots \times \text{domain}(A_n)$

□ **数据操纵的表示**：以关系为运算对象，用关系代数运算符可以组成一个关系代数表达式，用关系代数表达式来表示用户在关系模型上的数据操纵请求。

□ **数据操纵结果的表示**：在关系数据模型上进行数据操纵，其结果也构成一个关系。关系代数表达式的运算结果就是用户数据操纵请求的结果关系。



18. 关于两个关系之间的并、交、差运算

- ① 什么样的两个关系，可以在它们之间执行并、交或差运算？
- ② 如果在两个关系之间执行并、交或差运算，那么结果关系的关系模式和结果元组集合分别是什么？
- ③ 为什么说‘交’运算不是关系代数中的基本运算？
- ④ 在什么情况下需要使用‘差’运算？在使用‘差’运算时需要注意什么？
- ⑤ 请比较‘差’运算和‘ $\neq$ ’（不等）比较运算之间的区别。

① 当两个关系“具有相同的目 n （即两个关系都有 n 个属性）”且“相应的属性取自同一个域”时，才能在关系 R 和 S 之间执行并、交、差运算。

② 当两个关系 R 和 S 执行并、交、差运算时，结果关系的关系模式不变，与关系 R （或关系 S ）的关系模式一样。结果关系中的元组集合定义如下：

$$R \cup S = \{t \mid t \in R \vee t \in S\}$$

$$R \cap S = \{t \mid t \in R \wedge t \in S\}$$

$$R - S = \{t \mid t \in R \wedge t \notin S\}$$

$$S - R = \{t \mid t \in S \wedge t \notin R\}$$

③ 可以用‘差’运算来实现‘交’运算的运算功能：

$$R \cap S = R - (R - S) = S - (S - R)$$



18. 关于两个关系之间的并、交、差运算

- ① 什么样的两个关系，可以在它们之间执行并、交或差运算？
 - ② 如果在两个关系之间执行并、交或差运算，那么结果关系的关系模式和结果元组集合分别是什么？
 - ③ 为什么说‘交’运算不是关系代数中的基本运算？
 - ④ 在什么情况下需要使用‘差’运算？在使用‘差’运算时需要注意什么？
 - ⑤ 请比较‘差’运算和‘ $\neq$ ’（不等）比较运算之间的区别。
-
- ④ 当查询条件带有‘否定’语义，或者具有明显‘排它性’的时候，通常需要使用到‘差’运算；
使用‘差’运算时，在减数和被减数关系中，通常需要包含查询目标对象的码。
 - ⑤ 是两种不同类型的运算符，但在应用中又具有一定的‘相关性’。
 - (1) ‘差’运算是两个关系之间的集合运算，‘ $\neq$ ’是两个分量之间的逻辑比较运算，常用于构造数据查询的查询条件表达式；
 - (2) 在数据库查询中，需谨慎使用‘ $\neq$ ’比较运算；
 - (3) 即使查询条件带有‘否定’语义，通常也不是直接使用‘ $\neq$ ’比较运算来构造查询条件，而是使用两个子查询之间的‘差’运算来实现查询目标。（具体应用场景可参考后面的关系代数应用）



19. 关于两个关系之间的‘笛卡尔积’ (product) 运算 (参考答案)

- ① 什么样的两个关系，可以在它们之间执行笛卡尔积运算？
- ② 在两个关系之间执行笛卡尔积运算，结果关系的关系模式和结果元组集合分别是什么？
- ③ 笛卡尔积运算在关系数据库访问中有什么实际作用？

① 任意两个关系之间，都可以执行笛卡尔积运算。

② 设有一个 n 目关系 R 和 m 目关系 S ，那么笛卡尔积 $R \times S$ 的结果是一个 $(n+m)$ 目的关系，关系 R 和关系 S 的所有属性都会被投影到它们的笛卡尔积的结果关系中。

来自关系 R 的每一个元组 u 和来自关系 S 的每一个元组 v ，都会组合得到 $R \times S$ 结果关系中的一个元组，可形式化表示为： $R \times S = \{ (u, v) \mid u \in R \wedge v \in S \}$

③ 笛卡尔积是关系代数中的一个基本运算符。当数据查询需要使用到不同关系时，通常可以先通过笛卡尔积运算将多个关系合并成一个关系，然后再用单个关系上的选择和投影运算实现用户的数据查询请求，实现跨关系的数据访问。



第1组	$Q_1: \sigma_{pid = 'p01'}(orders)$	$Q_2: \sigma_{pid \neq 'p01'}(orders)$
-----	-------------------------------------	--

□ 查询结果集的语义是什么？

Q_1 : p01号商品的所有购买‘订单’

Q_2 : 除了p01号商品外，其他所有商品的购买‘订单’

□ 两个结果集的关系是什么？

$Q_1 \cap Q_2 = \text{空集}$

$Q_1 \cup Q_2 = \text{Orders}$



第2组	$Q_3: \pi_{cid}(\sigma_{pid = 'p01'}(orders))$	$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$
-----	--	---

□ 查询结果集的语义是什么？

Q_3 : 购买过p01号商品的‘顾客(编号)’

Q_4 : 购买过其他商品(pid不等于p01)的‘顾客(编号)’

□ 两个结果集的关系是什么？

- 两个结果集之间的关系，取决于当前orders表中的订单情况
- 两个结果集之间可能有交集，也可能没有交集，甚至于一个结果集包含着另一个结果集，一切皆有可能！
- $Q_3 \cup Q_4 = \pi_{cid}(orders)$

第20题--参考答案 (第3组)



第3组	Q ₅	$\pi_{cid}(\text{orders}) - \pi_{cid}(\sigma_{pid = 'p01'}(\text{orders}))$
	Q ₆	$\pi_{cid}(\text{orders}) - \pi_{cid}(\sigma_{pid \neq 'p01'}(\text{orders}))$

□ 查询结果集的语义是什么？

Q₅: 购买过商品但没有购买过p01号商品的‘顾客(编号)’

Q₆: 只购买过p01号商品的‘顾客(编号)’

□ 两个结果集的关系是什么？

$$Q_5 \cap Q_6 = \text{空集}$$

$$Q_5 \subseteq Q_4$$

$$Q_4: \pi_{cid}(\sigma_{pid \neq 'p01'}(\text{orders}))$$

$$Q_5 \cup Q_6 \subseteq \pi_{cid}(\text{orders})$$

$$Q_6 \subseteq Q_3$$

$$Q_3: \pi_{cid}(\sigma_{pid = 'p01'}(\text{orders}))$$



第4组	Q ₇	$\pi_{cid}(\text{customers}) - \pi_{cid}(\sigma_{pid = 'p01'}(\text{orders}))$
	Q ₈	$\pi_{cid}(\text{customers}) - \pi_{cid}(\sigma_{pid \neq 'p01'}(\text{orders}))$

□ 查询结果集的语义是什么？

Q₇: 没有购买过p01号商品的顾客 (包括“没有购买过任何商品”的顾客)

Q₈: “只购买过p01号商品”的顾客 和 “没有购买过任何商品” 的顾客

□ 两个结果集的关系是什么？

根据顾客的商品购买情况，将所有顾客划分为以下互不相交的四个子集：

- 没有购买过任何商品的顾客集合 S_1
- 只购买过p01这一种商品的顾客集合 S_2
- 没有购买过p01但购买过其他商品的顾客集合 S_3
- 既购买过p01又购买过其他商品的顾客集合 S_4

则: $Q_7 = S_1 \cup S_3$

$Q_7 \cap Q_8 = S_1$

$Q_8 = S_1 \cup S_2$

$Q_7 \cup Q_8 = S_1 \cup S_2 \cup S_3$

第20题--参考答案 (第5组)



第5组

Q_9	$orders - \sigma_{pid = 'p01'}(orders)$
Q_{10}	$orders - \sigma_{pid \neq 'p01'}(orders)$

□ 查询结果集的语义是什么？

Q_9 : 除了p01号商品之外，其他商品的所有销售订单

Q_{10} : p01号商品的所有销售订单

□ 两个结果集的关系是什么？

$Q_9 \cap Q_{10} = \text{空集}$

$Q_9 \cup Q_{10} = orders$

$Q_9 = Q_2$

$Q_{10} = Q_1$

$Q_1: \sigma_{pid = 'p01'}(orders)$
$Q_2: \sigma_{pid \neq 'p01'}(orders)$

(注: orders表中的pid不可能取空值)

第20题--参考答案 (第6组)



第6组	Q_{11}	$\pi_{cid}(orders - \sigma_{pid = 'p01'}(orders))$
	Q_{12}	$\pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$

□ 查询结果集的语义是什么？

Q_{11} : 购买过其他商品 (pid不等于'p01') 的顾客编号的集合

Q_{12} : 购买过商品'p01'的顾客编号的集合

□ 两个结果集的关系是什么？

就如同前面的 Q_3 和 Q_4 一样, Q_{11} 和 Q_{12} 的结果集之间也没有确定的相互关系, 一切取决于当前orders表中的订单情况。

- Q_{11} 是“购买过其他商品”的顾客集合, 其中的顾客有的可能也购买过'p01', 也有的可能没有购买过'p01';
- Q_{12} 是“购买过商品p01”的顾客集合, 其中的顾客可能也购买过其他商品, 也可能没有购买过其他商品。

第20题 -- 总结 & 思考 (注: '其他商品' 是指商品编号pid不等于'p01'的商品)



	查询表达式	目标对象	目标对象需要满足的条件
Q ₁	$\sigma_{pid='p01'}(orders)$	订单	'p01'号商品的销售订单
Q ₂	$\sigma_{pid \neq 'p01'}(orders)$	订单	其他商品的销售订单
Q ₃	$\pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客购买过'p01'号商品
Q ₄	$\pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客购买过其他商品
Q ₅	$\pi_{cid}(orders) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客没有购买过'p01', 但购买过其他商品
Q ₆	$\pi_{cid}(orders) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客只购买过'p01', 但没有购买过其他商品
Q ₇	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid='p01'}(orders))$	顾客	该顾客没有购买过'p01'号商品 (请注意与Q ₅ 的区别)
Q ₈	$\pi_{cid}(customers) - \pi_{cid}(\sigma_{pid \neq 'p01'}(orders))$	顾客	该顾客没有购买过其他商品 (请注意与Q ₆ 的区别)
Q ₉	$orders - \sigma_{pid='p01'}(orders)$	订单	其他商品的销售订单
Q ₁₀	$orders - \sigma_{pid \neq 'p01'}(orders)$	订单	'p01'号商品的销售订单
Q ₁₁	$\pi_{cid}(orders - \sigma_{pid='p01'}(orders))$	顾客	购买过其他商品的顾客 (请注意与Q ₅ 的区别)
Q ₁₂	$\pi_{cid}(orders - \sigma_{pid \neq 'p01'}(orders))$	顾客	购买过'p01'号商品的顾客 (请注意与Q ₆ 的区别)



21. 请用五种基本运算分别写出下列两个扩充运算的推导公式:

① ‘自然联接’运算 ② ‘除’运算

① ‘自然联接’运算的推导公式

➤ 设有两个关系 R 和 S , 其关系模式如下, $C_1, C_2, \dots, C_k$ 是它们两者的公共属性

$$\text{Head}(R) = \{ A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \}$$

$$\text{Head}(S) = \{ B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k \}$$

➤ 结果关系的关系模式是:

$$\text{Head}(R \bowtie S) = \text{Head}(R) \cup \text{Head}(S) = \{ A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k, B_1, B_2, \dots, B_m \}$$

➤ 结果关系的推导公式是: $R \bowtie S = \pi_{A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k, B_1, B_2, \dots, B_m} (\sigma_{R.C_1=S.C_1 \wedge R.C_2=S.C_2 \wedge \dots \wedge R.C_k=S.C_k} (R \times S))$

② ‘除’运算的推导公式

➤ 设有两个关系 R 和 S , 其关系模式如下:

$$\text{Head}(R) = \{ A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m \}$$

$$\text{Head}(S) = \{ B_1, B_2, \dots, B_m \}$$

➤ 结果关系的关系模式是: $\text{Head}(R \div S) = \text{Head}(R) - \text{Head}(S) = \{ A_1, A_2, \dots, A_n \}$

➤ 结果关系的推导公式是: $R \div S = \pi_{A_1, A_2, \dots, A_n} (R) - \pi_{A_1, A_2, \dots, A_n} ((\pi_{A_1, A_2, \dots, A_n} (R) \times S) - R)$



22. 理解笛卡尔积、 θ -联接、自然联接三者之间的联系和区别，并掌握：①在何种场景下适合使用自然联接？②如何正确使用‘自然联接’运算。

都是关系的合并运算，都可以用来实现两个关系的合并。其中，‘笛卡尔积’是基本运算，‘ θ -联接’和‘自然联接’则是扩充运算，它们之间的联系和区别是：

❑ 笛卡尔积

- 是实现跨不同关系表进行数据访问的基础，在任意两个关系之间都可以用‘笛卡尔积’运算实现它们的合并；
- 设有一个 n 元关系 R 和 m 元关系 S ，笛卡尔积 $R \times S$ 的结果是一个 $(n+m)$ 元的关系，结果元组集合可形式化表示为： $R \times S = \{ (u, v) \mid u \in R \wedge v \in S \}$
- 在笛卡尔积的结果关系中，通常存在着很多无意义结果元组，需要通过后续选择运算过滤掉。

❑ θ -联接

- 通过用户定义的联接条件 F 来实现两个关系的合并；
- 可以把一次‘ θ -联接’运算看成是一次“笛卡尔积”+一次“选择运算”， θ -联接结果关系的关系模式与笛卡尔积的结果关系相同，结果元组集合则是笛卡尔积结果集的一个子集， θ -联接的推导公式如下： $R \bowtie_F S = \sigma_F(R \times S)$

❑ 自然联接

- 将两个关系通过它们的“所有同名属性的相等比较”来进行合并的一种关系运算；
- 自然联接结果关系的关系模式与笛卡尔积、 θ -联接的结果关系的模式不同；
- 也可以用“笛卡尔积+选择+投影”运算来实现自然联结的查询结果。（推导公式见第1题）



(cont.) 执行关系之间的联接查询时，一般通用方法是通过‘笛卡尔积’或者‘ θ -联接’来实现，更常用的方法是‘自然联接’。

❑ 一般方法：笛卡尔积 or θ -联接

- 当两个关系之间“不存在同名属性”，或者“联接条件不是基于同名属性的相等比较”时，需要采用‘笛卡尔积’或者‘ θ -联接’实现关系联接查询。
- 在笛卡尔积和 θ -联接的结果关系中可能存在同名属性，使用时需要加以区别。

❑ 常用方法：自然联接

- 自然联接的联接条件是隐含在运算符中的（所有同名属性的相等比较）
- 如果在两个关系之间“存在多对同名属性，而本次查询又不需要‘所有’的同名属性都相等”，此时有两种选择：
 - ① 不使用自然联结，转而采用笛卡尔积或 θ -联接来实现关系的合并；
 - ② 先对其中的一个关系执行投影运算，过滤掉其中不需要进行相等比较的那些同名属性，然后再使用自然联接运算。

❑ 难点（关系的自联接）：‘自联接’是指一个关系自己和自己进行联接运算，这个通常无法使用‘自然联接’，只能使用 θ -联接或者笛卡尔积。一般方法如下：

- 使用赋值运算定义‘同质不同名’的两个中间关系（元组集合相同，但关系名不同），当然也可以对中间关系中的属性进行重命名；
- 然后，再使用笛卡尔积或 θ -联接实现两个中间关系的合并；
- 也可以通过有目的地对中间关系中属性进行重命名，然后用自然联结实现中间关系的合并。

23. 如何理解‘除’运算的语义？请解释其推导公式的含义，并掌握如何正确书写‘除’运算表达式。

□ 关于‘除’运算的结果元组的语义：设在关系R和S之间可以执行‘除’运算 $R \div S$ ，结果关系为T。如果一个元组 $t \in T$ ，那么对于除数关系S中的每一个元组s，都有 $(t, s) \in R$

□ 关于‘除’运算的推导公式是 $R \div S = \pi_{A_1, A_2, \dots, A_n}(R) - \pi_{A_1, A_2, \dots, A_n}((\pi_{A_1, A_2, \dots, A_n}(R) \times S) - R)$

□ 其中每一步的计算目的如下：

1) $T_{max} := \pi_{A_1, A_2, \dots, A_n}(R)$ // T_{max} 是最大可能的结果元组集合

2) $R_{max} := T_{max} \times S$ // R_{max} 与关系R是同类关系，如果 $R_{max} \subseteq R$ ，那么 $R \div S$ 的结果关系就是 T_{max}

3) $T_1 := R_{max} - R$ // 事实上，会有很多 R_{max} 中的元组没有出现在关系R中

4) $T_2 := \pi_{A_1, A_2, \dots, A_n}(T_1)$ // T_2 就是关系 T_{max} 中不满足除运算结果要求的那些元组
// 即：对于关系 T_2 中的任一个元组q，至少能在关系S中找到一个元组s，
// 并且由元组q和s所构成的元组(q,s)不在关系R中出现。

5) $R \div S := T_{max} - T_2$

□ 在使用‘除’运算时，正确的使用方法如下：

- 我们将查询结果中的客观对象称为‘目标对象’，用于定义查询条件的客观对象称为‘条件对象’；
- 在被除数关系中必须包含目标对象和条件对象的关键字；
- 除数关系中只含条件对象的关键字；
- 被除数和除数关系中不能含其它‘不必要’的多余属性。



24. 理解笛卡儿积/ θ -联接/自然联接和‘除’运算之间的区别，并回答：

- ① 在什么情况下适合应用除运算而不是联接运算？请举一例并写出使用除法和不使用除法的两种查询表达式。
- ② 请列举一个与情况①相似但又不能使用除法的例子，并写出其查询表达式。

- 我们将查询结果中的客观对象称为‘目标对象’，用于定义查询条件的客观对象称为‘条件对象’。在判断某个目标对象 t （结果元组 t ）是否属于结果关系时，
- 如果只需要从条件对象集合中找到一个条件对象 c （元组 c ），并且目标对象 t 和条件对象 c 之间满足查询条件，那么就直接使用‘联接’运算（包括笛卡尔积、 θ -联接和自然联接）；
 - 如果需要条件对象集合中的所有条件对象 c 和目标对象 t 之间都满足查询条件，那么就使用‘除’运算。

顾客 C (<u>cid</u> , cname, city, discent)	商品 P (<u>pid</u> , pname, city, stqty, price)
供应商 A (<u>aid</u> , aname, city, percent)	订单 O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

① 例：向‘南京’市的所有顾客都销售过商品的供应商的编号；

- (使用除运算): $\pi_{aid, cid}(O) \div \pi_{cid}(\sigma_{city='南京'}(C))$
- (不使用除运算): $\pi_{aid}(O) - \pi_{aid}(\pi_{aid}(A) \times \pi_{cid}(\sigma_{city='南京'}(C))) - \pi_{aid, cid}(O)$

② 例：通过顾客自己所在城市中的所有供应商都购买过商品的顾客的编号；

- 本题不能使用除运算来表示，正确表示方法是: $\pi_{cid}(O) - \pi_{cid}(\pi_{cid, aid}(A \bowtie C) - \pi_{cid, aid}(O))$



25. 什么是关系的自联接？请列举一个使用自联接的例子，并写出查询表达式。

所谓的‘自联接’，就是指一个关系自己与自己进行‘联接’查询的情况。例如：在每一个城市中查询 percent 值最高的供应商，结果返回这些供应商所在的城市名称、供应商编号及其 percent 值。该查询的表示方法是：令 $M := A, N := A$ ，查询表达式为：

$$\pi_{city, aid, percent}(A) - \pi_{M.city, M.aid, M.percent}(\sigma_{M.city=N.city \wedge M.percent < N.percent}(M \times N))$$

顾客 C (<u>cid</u> , cname, city, discent)	商品 P (<u>pid</u> , pname, city, stqty, price)
供应商 A (<u>aid</u> , aname, city, percent)	订单 O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

26. 什么是外联接？请列举一个使用外联接的例子，并写出查询表达式。

所谓的‘外联接’，是一种特殊的关系联接运算。外联接结果关系的关系模式与它们之间自然联接的结果关系模式相同，外联接的结果元组集合既包含着它们自然联接的结果元组，也包含着联接对象关系中那些没有出现在自然联接结果集中的元组。在外联接的结果关系上，使用关系代数运算可以重构原来的联接对象关系。

例如：查询顾客与供应商位于同一个城市中的订单，结果返回城市名称、顾客编号、供应商编号、订单编号。结果集中也要包含顾客和供应商位于同一个城市、但他们之间没有产生过销售订单的情况（返回的订单编号为‘空值’）。

我们用‘left-outer-join’表示‘左外联接’运算，该查询可表示为：

$$\pi_{C.city, C.cid, A.aid, O.ordno}((C \bowtie A) \text{ left-outer-join } O)$$



C (<u>cid</u> , cname, city, discent)	A (<u>aid</u> , aname, city, percent)
P (<u>pid</u> , pname, city, stqty, price)	O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

27-(1) 查询满足条件的顾客的编号cid:

① 只购买过 p01 和 p02 两种商品;

$$\pi_{cid}(\sigma_{pid = 'p01'}(O)) \cap \pi_{cid}(\sigma_{pid = 'p02'}(O)) - \pi_{cid}(\sigma_{pid \neq 'p01' \wedge pid \neq 'p02'}(O))$$

② “只购买过商品p01” 或者 “只购买过商品p02”;

$$(\pi_{cid}(\sigma_{pid = 'p01'}(O)) - \pi_{cid}(\sigma_{pid \neq 'p01'}(O))) \cup (\pi_{cid}(\sigma_{pid = 'p02'}(O)) - \pi_{cid}(\sigma_{pid \neq 'p02'}(O)))$$

③ 只购买过一种商品。

令R:=O, S:=O, 该查询可表示为: $\pi_{cid}(O) - \pi_{R.cid}(\sigma_{R.cid=S.cid \wedge R.pid \neq .pid}(R \times S))$

27-(2) 查询满足条件的供应商的名字aname: 没有销售过商品;

$$\pi_{aname}((\pi_{aid}(A) - \pi_{aid}(O)) \bowtie A)$$



C (<u>cid</u> , cname, city, discent)	A (<u>aid</u> , aname, city, percent)
P (<u>pid</u> , pname, city, stqty, price)	O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

27-(3) 查询满足条件的供应商的编号aid:

① 只向供应商自己所在城市中的顾客销售过商品;

$$\pi_{aid}(O) - \pi_{A.aid}(\sigma_{A.aid = O.aid \wedge O.cid = C.cid \wedge A.city \neq C.city}(A \times O \times C))$$

② 在所有有顾客的城市中都向客户销售过商品;

答案1 (仅使用基本运算符)

$$\pi_{aid}(O) - \pi_{aid}(\pi_{A.aid, C.city}(A \times C) - \pi_{O.aid, C.city}(\sigma_{O.cid = C.cid}(O \times C)))$$

答案2 (使用到自然连接运算) $\pi_{aid}(O) - \pi_{aid}(\pi_{A.aid, C.city}(A \times C) - \pi_{O.aid, C.city}(O \bowtie C))$

答案3 (使用到除法运算) $\pi_{O.aid, C.city}(O \bowtie C) \div \pi_{city}(C)$

③ 向供应商自己所在城市中的所有顾客都销售过商品。

$$\pi_{aid}(O) - \pi_{aid}(\pi_{aid, cid}(A \bowtie C) - \pi_{aid, cid}(O))$$



C (<u>cid</u> , cname, city, discent)	A (<u>aid</u> , aname, city, percent)
P (<u>pid</u> , pname, city, stqty, price)	O (<u>ordno</u> , orddate, cid, aid, pid, qty, dols)

27-(4) 在每一个供应商自己的所有订单中，查询销售金额dols最高的订单，结果返回供应商编号、该供应商的销售金额最高订单的订单编号和销售金额。

令 $M:=O$ $N:=O$ ，该查询可表示为：

$$\pi_{aid, ordno, dols} (O) - \pi_{M.aid, M.ordno, M.dols} (\sigma_{M.aid=N.aid \wedge M.dols < N.dols} (M \times N))$$



27-(5) 查询每一个顾客的第一份订单和最后一份订单（在第①和第②小题中，不考虑没有订单和只有一份订单的顾客；在第③小题中，需要返回所有顾客的查询结果）

① 结果关系中只有 **cid, ordno, orddate** 三个属性；

➤ 查询每个客户的最后一份订单 E

(step 1) 查询每个客户的‘历史订单’（每个客户在最后一份订单之前的订单）

令 $O_1 := O, O_2 := O, M := \pi_{O1.cid, O1.ordno, O1.orddate}(\sigma_{O1.cid=O2.cid \wedge O1.ordno < O2.ordno}(O_1 \times O_2))$

(step 2) 查询每个客户的最后一份订单 E: $E(cid, lastord, lastdate) := \pi_{cid, ordno, orddate}(O) - M$

➤ 使用与上面类似的方法，查询每个客户的第一份订单 F

(step 3) 查询每个客户的‘后继订单’（每个客户在第一份订单之后的订单）

令 $O_1 := O, O_2 := O, N := \pi_{O2.cid, O2.ordno, O2.orddate}(\sigma_{O1.cid=O2.cid \wedge O1.ordno < O2.ordno}(O_1 \times O_2))$

(step 4) 查询每个客户的第一份订单 F: $F(cid, ordno, orddate) := \pi_{cid, ordno, orddate}(O) - N$

① 答：令 $lastorder(cid, ordno, orddate) := E$ ，本题查询可表示为： **$lastorder \cup F$**

说明：本题查询结果中，不含没有订单的顾客；如果是只有一份订单的顾客，结果集中只返回一条结果元组。



27-(5) 查询每一个顾客的第一份订单和最后一份订单（在第①和第②小题中，不考虑没有订单和只有一份订单的顾客；在第③小题中，需要返回所有顾客的查询结果）

② 结果关系中有顾客编号cid, 第一份订单的ordno和orddate, 最后一份订单的ordno和orddate等五个属性；

➤ 查询每个客户的最后一份订单 E

(step 1) 查询每个客户的‘历史订单’（每个客户在最后一份订单之前的订单）

令 $O_1 := O$, $O_2 := O$, $M := \pi_{O1.cid, O1.ordno, O1.orddate}(\sigma_{O1.cid=O2.cid \wedge O1.ordno < O2.ordno}(O_1 \times O_2))$

(step 2) 查询每个客户的最后一份订单 E: $E(cid, lastord, lastdate) := \pi_{cid, ordno, orddate}(O) - M$

➤ 使用与上面类似的方法，查询每个客户的第一份订单 F

(step 3) 查询每个客户的‘后继订单’（每个客户在第一份订单之后的订单）

令 $O_1 := O$, $O_2 := O$, $N := \pi_{O2.cid, O2.ordno, O2.orddate}(\sigma_{O1.cid=O2.cid \wedge O1.ordno < O2.ordno}(O_1 \times O_2))$

(step 4) 查询每个客户的第一份订单 F: $F(cid, ordno, orddate) := \pi_{cid, ordno, orddate}(O) - N$

② 每个客户返回一条元组，包括 cid, 第一份订单ordno和orddate, 最后一份订单的lastord和lastdate, 本题查询可表示为：

$\pi_{F.cid, F.ordno, F.orddate, E.lastord, E.lastdate}(\sigma_{F.cid=E.cid \text{ and } F.ordno \neq E.lastord}(F \times E))$



27-(5) 查询每一个顾客的第一份订单和最后一份订单（在第①和第②小题中，不考虑没有订单和只有一份订单的顾客；在第③小题中，需要返回所有顾客的查询结果）

③ 结果关系的关系模式同上面的第②小题，每一个顾客返回一条查询结果。如果是没有订单的顾客，结果元组上只有该顾客的编号；如果是只有一份订单的顾客，那么这份订单就作为该顾客的最后一份订单，他的第一份订单信息为空。（提示：使用外联接）

(step 1) 查询每个客户的‘历史订单’（每个客户在最后一份订单之前的订单）

令 $O_1 := O$, $O_2 := O$, $H := \pi_{O1.cid, O1.ordno, O1.orddate}(\sigma_{O1.cid=O2.cid \wedge O1.ordno < O2.ordno}(O_1 \times O_2))$

(step 2) 查询每个客户的最后一份订单E: $E(cid, lastord, lastdate) := \pi_{cid, ordno, orddate}(O) - H$

(step 3) 查询每个客户的‘中间订单’（在历史订单H中，查询每个客户在第一份订单之后的订单）

令 $H_1 := H$, $H_2 := H$, $M := \pi_{H2.cid, H2.ordno, H2.orddate}(\sigma_{H1.cid=H2.cid \text{ and } H1.ordno < H2.ordno}(H_1 \times H_2))$

(step 4) 查询每个客户的第一份订单F: $F(cid, ordno, orddate) := H - M$

③ 用 left-outer-join 表示‘左外联接’运算，本题查询可表示为：

$(\pi_{cid}(C) \text{ left-outer-join } E) \text{ left-outer-join } F$



关系	属性集	关系模式
学生	<u>学号</u> , 学生姓名, 就读院系, 年级	S (<u>sno</u> , sname, dept, grade)
课程	<u>课程号</u> , 课程名, 开课院系, 课程类型	C (<u>cno</u> , cname, dept, opt)
教师	<u>教师工号</u> , 教师姓名, 工作院系	T (<u>tno</u> , tname, dept)
选课	<u>学号</u> , <u>课程号</u> , 授课教师工号, 成绩, 修读年份	L (<u>sno</u> , <u>cno</u> , tno, score, years)

28. 设有一个如上所示的学生选课数据库，其中：带下划线的属性是关键字；课程类型分为‘必修’和‘选修’；同一门课同一个学生只能有一条选课记录；成绩采用百分制。请用关系代数写出下述查询。查询满足下述条件的学生的学号和姓名：

① 只修读过自己就读院系开设的课程；

解： $\pi_{sno, sname} ((\pi_{sno} (L) - \pi_{sno} (\sigma_{S.sno=L.sno \wedge C.cno=L.cno \wedge S.dept \neq C.dept} (L \times S \times C))) \bowtie S)$

② 所有的修读课程成绩都合格（成绩 ≥ 60 ）；

解： $\pi_{sno, sname} ((\pi_{sno} (L) - \pi_{sno} (\sigma_{score < 60} (L))) \bowtie S)$



关系	属性集	关系模式
学生	<u>学号</u> , 学生姓名, 就读院系, 年级	S (<u>sno</u> , sname, dept, grade)
课程	<u>课程号</u> , 课程名, 开课院系, 课程类型	C (<u>cno</u> , cname, dept, opt)
教师	<u>教师工号</u> , 教师姓名, 工作院系	T (<u>tno</u> , tname, dept)
选课	<u>学号</u> , <u>课程号</u> , 授课教师工号, 成绩, 修读年份	L (<u>sno</u> , <u>cno</u> , tno, score, years)

③ 修读过‘计算机’系开设的所有‘必修’课程；

解1（不使用‘除’运算）：

$$\pi_{sno,sname}((\pi_{sno}(L) - \pi_{sno}((\pi_{sno}(S) \times \pi_{cno}(\sigma_{dept='计算机' \wedge opt='必修'}(C))) - \pi_{sno,cno}(L)))) \bowtie S)$$

解2（使用‘除’运算）： $\pi_{sno,sname,cno}(L \bowtie S) \div \pi_{cno}(\sigma_{dept='计算机' \wedge opt='必修'}(C))$

④ 修读了自己就读院系开设的所有‘必修’课程且在这些必修课程上的成绩都合格；

解（本题不能使用‘除’运算）：

$$\pi_{sno,sname}((\pi_{sno}(L) - \pi_{sno}((\pi_{S.sno,C.cno}(\sigma_{opt='必修'}(S \bowtie C))) - \pi_{sno,cno}(\sigma_{score \geq 60}(L)))) \bowtie S)$$



关系	属性集	关系模式
学生	<u>学号</u> , 学生姓名, 就读院系, 年级	S (<u>sno</u> , sname, dept, grade)
课程	<u>课程号</u> , 课程名, 开课院系, 课程类型	C (<u>cno</u> , cname, dept, opt)
教师	<u>教师工号</u> , 教师姓名, 工作院系	T (<u>tno</u> , tname, dept)
选课	<u>学号</u> , <u>课程号</u> , 授课教师工号, 成绩, 修读年份	L (<u>sno</u> , <u>cno</u> , tno, score, years)

- ⑤ 2019级并且还没有修读通过自己就读院系开设的所有‘必修’课程。（存在自己就读院系开设的‘必修’课程，该同学还没有修读，或者虽然修读了但成绩没达到合格。）

解（本题不能使用‘除’运算）：

$$\pi_{sno, sname}((\pi_{S.sno, C.cno}(\sigma_{grade=2019 \wedge opt='必修'}(S \bowtie C)) - \pi_{sno, cno}(\sigma_{score \geq 60}(L))) \bowtie S)$$